

TAREA PROGRAMATIVA

DESARROLLO DE UN SISTEMA DE INCIDENCIAS

Aplicación de Xanpan e Inteligencia Artificial

Curso	[ITI-822] METODOLOGIAS AGILES DE DESARROLLO DE SOFTWARE
Grupo	SC-01 SC-02
Docente	Andrés Joseph Jiménez Leandro
Modalidad	Parejas/dúos de exactamente dos estudiantes
Plataforma de Entrega	Campus virtual y repositorio

Propósito General

Construir un producto de software pequeño pero verificable, utilizando Kanban para gobernar el flujo de trabajo, prácticas de XP para incorporar calidad técnica y un uso transparente y crítico de herramientas de inteligencia artificial.

1. Descripción General

La organización requiere una aplicación denominada **HelpDesk Flow**, orientada al registro, priorización, atención, validación y cierre de incidencias técnicas. El producto podrá desarrollarse como aplicación de consola; con interfaz gráfica y la base de datos para la persistencia de datos.

Enfoque de Evaluación

La complejidad principal no consiste en construir numerosas pantallas, sino en demostrar un flujo de trabajo disciplinado, pruebas automatizadas, integración frecuente, refactorización segura, trazabilidad del repositorio y capacidad de adaptación a un cambio de requerimiento.

1.1 Modalidad y Tecnologías

- La tarea se desarrolla exclusivamente en parejas o dúos de exactamente dos estudiantes.
- Lenguaje: Java 17 o superior.
- Framework de pruebas: JUnit 5 ó JMeter.
- Repositorio Git con acceso para el docente.
- Integración continua mediante GitHub Actions o una herramienta equivalente.
- Tablero Kanban en GitHub Projects, Trello, Jira, ClickUp, Linear.App, Azure Boards (de Azure DevOps), Kanban Tool, Asana, Notion, Pipefy, Wekan, Kanboard, Orangescrum, o una plataforma similar.

1.2 Información mínima de cada incidencia

- Identificador único.
- Título.
- Descripción.
- Categoría.
- Impacto.
- Urgencia.
- Prioridad calculada.
- Estado.
- Fecha de creación.
- Fecha de cierre.
- Descripción de la solución aplicada.

2. Historias de usuario obligatorias

HU-01. Registrar una incidencia

Como usuario, deseo registrar una incidencia indicando su título, descripción, impacto, urgencia y categoría, para que el equipo técnico pueda atenderla.

Criterios de aceptación

- El título no puede estar vacío.
- La descripción debe contener al menos diez caracteres.
- El impacto debe ser BAJO, MEDIO o ALTO.

- La urgencia debe ser BAJA, MEDIA o ALTA.
- El sistema debe generar un identificador único.

HU-02. Calcular automáticamente la prioridad

Como encargado de soporte, deseo que el sistema calcule la prioridad de la incidencia para evitar decisiones arbitrarias.

Reglas iniciales

Impacto	Urgencia	Prioridad
Alto	Alta	Crítica
Alto	Media o baja	Alta
Medio o bajo	Alta	Alta
Cualquier otra combinación	Cualquier otra combinación	Normal

HU-03. Gestionar el flujo de la incidencia

Como técnico, deseo cambiar el estado de una incidencia para representar su avance real.

Estados permitidos

REGISTRADA → LISTA → EN_DESARROLLO → EN_VALIDACION → FINALIZADA

Restricciones

- El sistema debe impedir saltos o retrocesos no autorizados entre estados.
- REGISTRADA → FINALIZADA es una transición inválida.
- FINALIZADA → EN_DESARROLLO es una transición inválida.
- Una incidencia no puede pasar a FINALIZADA si no posee una descripción de la solución aplicada.

HU-04. Consultar y filtrar incidencias

El sistema debe permitir:

- Mostrar todas las incidencias.
- Buscar una incidencia por identificador.
- Filtrar por estado.
- Filtrar por prioridad.
- Mostrar solamente incidencias abiertas.
- Mostrar solamente incidencias finalizadas.

HU-05. Generar métricas básicas

El sistema deberá mostrar:

- Cantidad total de incidencias.
- Cantidad de incidencias finalizadas.
- Cantidad de incidencias abiertas.
- Throughput: incidencias terminadas durante el periodo registrado.
- Lead time promedio de las incidencias terminadas.
- Cantidad de incidencias por prioridad.

3. Cambio obligatorio de requerimiento

Nuevo Requerimiento

La organización incorpora una clase de servicio denominada EXPEDITE. Una incidencia crítica podrá marcarse como urgente y recibir atención prioritaria. Solamente podrá existir una incidencia EXPEDITE en desarrollo o validación simultáneamente.

Para incorporar el cambio, la pareja deberá:

1. Crear una nueva tarjeta en el backlog.
2. Definir criterios de aceptación verificables.
3. Escribir primero las pruebas asociadas al cambio.
4. Adaptar el diseño existente sin eliminar el comportamiento previamente validado.
5. Mantener en funcionamiento todas las pruebas anteriores.
6. Integrar el cambio sin reescribir innecesariamente el sistema.

4. Aplicación obligatoria de Kanban

La dupla (pareja/dúo) deberá administrar el trabajo mediante un tablero que represente el estado real del producto y permita observar el flujo, los bloqueos y los límites de trabajo en progreso.

4.1 Columnas mínimas

Opciones / Backlog	Preparado	En desarrollo	Validación	Hecho
--------------------	-----------	---------------	------------	-------

4.2 Límites WIP

Columna	Límite de trabajo en progreso
Preparado	3
En desarrollo	1 por pareja
Validación	1
Hecho	Sin límite

Regla de Flujo

Cuando una columna alcanza su límite WIP, no se inicia trabajo nuevo en esa etapa. La prioridad es terminar, desbloquear, revisar o ayudar a avanzar lo que ya está comprometido.

4.3 Políticas Explícitas

Movimiento	Condiciones obligatorias
Entrada a Preparado	Descripción comprensible; criterios de aceptación; tamaño manejable; pruebas principales identificadas.
Entrada a Validación	El código compila; las pruebas pasan localmente; existe revisión del otro integrante; no hay cambios relevantes sin confirmar.
Entrada a Hecho	La integración continua está en verde; criterios de aceptación verificados; código integrado en la rama principal; documentación actualizada.

No se permite mover todas las tarjetas a “Hecho” únicamente al final. El historial del tablero debe evidenciar avance progresivo y coherente con el repositorio.

5. Aplicación obligatoria de Extreme Programming (XP)

5.1 Test-Driven Development (TDD)

Se requieren como mínimo las siguientes pruebas automatizadas:

- Ocho pruebas unitarias en total.
- Al menos dos pruebas funcionales en total.
- Pruebas para las reglas de cálculo de prioridad.
- Pruebas para transiciones válidas.
- Pruebas para transiciones inválidas.
- Prueba que demuestre que no puede cerrarse una incidencia sin solución.
- Pruebas específicas para el cambio EXPEDITE.

Evidencia TDD

El historial de commits debe mostrar evidencia razonable del ciclo RED → GREEN → REFACTOR. No basta con escribir todas las pruebas después de finalizar la implementación.

5.2 Programación en pareja

Ambos integrantes deberán alternar responsabilidades y participar activamente en el desarrollo:

- **Driver:** escribe el código y ejecuta los cambios inmediatos.
- **Navigator:** analiza, cuestiona, revisa casos límite y orienta las decisiones de diseño.
- **Ping-Pong TDD:** un integrante escribe una prueba, el otro implementa lo mínimo para hacerla pasar y luego intercambian roles.
- **Revisión cercana:** cuando no sea viable trabajar simultáneamente, el segundo integrante revisa el cambio con contexto reciente.

5.3 Integración continua

El repositorio deberá contener una automatización que:

- Compile el proyecto.
- Ejecute las pruebas.
- Falle cuando una prueba no pasa.
- Se ejecute con cada push o pull request.
- Mantenga visible el estado de la rama principal.

5.4 Refactorización

La pareja deberá documentar al menos una refactorización real, por ejemplo:

- Extraer una clase o método.
- Eliminar código duplicado.
- Cambiar nombres poco claros.
- Separar el cálculo de prioridad de la gestión de incidencias.
- Sustituir condicionales extensos por un diseño más mantenible.
- Dividir un componente con demasiadas responsabilidades.

La evidencia deberá explicar:

1. Problema encontrado.
2. Cambio realizado.
3. Pruebas que protegieron la refactorización.
4. Resultado obtenido.

6. Uso permitido de inteligencia artificial

La inteligencia artificial puede emplearse como asistencia técnica. La pareja conserva la responsabilidad sobre la corrección, seguridad, autoría y comprensión del producto entregado.

6.1 Usos permitidos

- Proponer criterios de aceptación y casos de prueba.
- Explicar errores de compilación o ejecución.
- Revisar código y sugerir nombres de clases o métodos.
- Proponer refactorizaciones.
- Generar una primera versión de una prueba que luego sea revisada.
- Ayudar a configurar la integración continua.
- Mejorar la documentación del repositorio.
- Comparar alternativas de diseño.

6.2 Prácticas no aceptables

- Solicitar una solución completa y entregarla sin revisión.
- Copiar código que ninguno de los integrantes pueda explicar.
- Fabricar commits, capturas, pruebas o evidencias.
- Afirmar que se aplicó TDD cuando las pruebas fueron creadas solamente al final.
- Ocultar errores o recomendaciones incorrectas producidas por la IA.

6.3 Bitácora obligatoria de IA

El repositorio deberá incluir el archivo IA-LOG.md con un mínimo de tres interacciones relevantes.

Fecha	Herramienta	Objetivo	Resultado usado	Verificación	Cambios humanos

- Al menos una respuesta de IA que haya sido modificada por la pareja.
- Al menos una sugerencia que haya sido rechazada.
- La razón técnica por la que la sugerencia fue rechazada.
- La forma en que se verificó el resultado finalmente utilizado.

7. Repositorio, commits y trazabilidad

Requisito diario de commits

Durante todo el periodo previo a la entrega, el repositorio debe registrar al menos un commit significativo por cada día calendario. Los commits vacíos, artificiales o realizados únicamente para cumplir la cantidad no serán considerados evidencia válida.

Además, se aplican las siguientes condiciones:

- Los dos integrantes deben aparecer como autores de commits significativos y distribuidos a lo largo del proceso.
- Los mensajes de commit deben describir el cambio realizado.
- Se deben preferir cambios pequeños y verificables frente a un único commit masivo.
- Cada historia o corrección relevante debe poder relacionarse con una tarjeta del tablero.
- La rama principal debe conservarse compilable y con las pruebas en verde.
- El historial no debe reescribirse para ocultar errores, fallos o participación desigual.

Ejemplos de mensajes aceptables (de preferencia en inglés, si no en español):

`test: add failing tests for critical priority calculation`
`feat: implement automatic incident priority`
`refactor: extract incident state transition validator`
`fix: prevent closing incidents without solution`
`ci: execute JUnit tests on pull requests`

8. Entregables

8.1 Repositorio

```
/src
/tests
/.github/workflows
README.md
IA-LOG.md
RETROSPECTIVA.md
```

8.2 Tablero Kanban (link o acceso al recurso)

- Historias de usuario.
- Tareas técnicas.
- Defectos encontrados.
- Cambio de requerimiento.
- Límites WIP visibles.
- Movimiento progresivo de tarjetas.

8.3 README.md

- Descripción del sistema.
- Nombres de los dos integrantes.
- Requisitos de ejecución.
- Instrucciones para compilar.
- Instrucciones para ejecutar las pruebas.
- Enlace al tablero.
- Decisiones principales de diseño.
- Estado de la integración continua.

8.4 Retrospectiva

El archivo RETROSPECTIVA.md deberá contener entre 400 y 700 palabras y responder:

1. ¿Qué aportó Kanban al trabajo de la pareja?
2. ¿Qué dificultad generó el límite WIP?

3. ¿Qué errores fueron detectados mediante TDD?
4. ¿Qué parte del código fue refactorizada?
5. ¿Cómo afectó el cambio de requerimiento?
6. ¿En qué ayudó la IA?
7. ¿En qué se equivocó o fue insuficiente la IA?
8. ¿Qué cambiarían en una siguiente versión?

8.5 Demostración técnica

Durante la revisión o defensa, la pareja deberá:

- Ejecutar el programa.
- Mostrar una regla de negocio.
- Ejecutar las pruebas.
- Mostrar el estado de la integración continua.
- Explicar una tarjeta del tablero.
- Mostrar la refactorización realizada.
- Explicar una interacción relevante con IA.
- Responder preguntas técnicas individuales.

9. Plan de trabajo esperado

La siguiente secuencia representa el trabajo esperado y puede ajustarse mientras se respeten las dependencias técnicas, el flujo Kanban y la trazabilidad del repositorio.

Etapas	Trabajo esperado
Preparación	Crear el repositorio, configurar el tablero, redactar historias y criterios de aceptación, y generar el proyecto base.
Primer incremento	Implementar el registro de incidencias y el cálculo de prioridad mediante TDD.
Flujo funcional	Implementar estados, transiciones, búsquedas y filtros.
Adaptación	Incorporar el cambio EXPEDITE, sus políticas y sus pruebas.
Calidad técnica	Completar métricas, integración continua y refactorización.
Validación	Corregir defectos, verificar criterios de aceptación y actualizar documentación.
Cierre	Preparar la entrega, revisar evidencias, completar la retrospectiva.
Demostración Técnica	Realizar la demostración técnica funcional de la aplicación en la siguiente clase delante del grupo.

10. Rúbrica de evaluación

Criterio	Puntos	Evidencia esperada
Funcionamiento general del sistema	20	Las historias obligatorias operan correctamente y el sistema puede demostrarse.
Reglas de negocio y manejo de errores	10	Validaciones, transiciones y restricciones se aplican de manera consistente.
TDD y calidad de las pruebas	15	Pruebas pertinentes, evidencia de RED-GREEN-REFACTOR y cobertura de casos límite.

Criterio	Puntos	Evidencia esperada
Refactorización y diseño del código	10	Código legible, responsabilidades claras y mejora documentada sin romper comportamiento.
Kanban, límites WIP y políticas	15	Tablero actualizado progresivamente, flujo real y políticas explícitas respetadas.
Integración continua e historial Git	10	Pipeline funcional, commits diarios significativos y participación verificable de ambos integrantes.
Adaptación al cambio EXPEDITE	10	El nuevo requerimiento se incorpora con pruebas, diseño coherente y regresión controlada.
Uso crítico y transparente de IA	5	Bitácora completa, verificación, modificación y rechazo razonado de sugerencias.
Documentación, retrospectiva y defensa	5	Entregables completos y dominio técnico demostrable por ambos integrantes.
TOTAL	100	

11. Condiciones de Calificación

Condición	Aplicación
Pruebas ausentes	Un programa funcional sin pruebas automatizadas no puede obtener más de 70 puntos.
TDD no demostrable	Las pruebas creadas únicamente al final reducen el puntaje del criterio de TDD.
Tablero retrospectivo	Un tablero actualizado solamente antes de la entrega no demuestra la aplicación de Kanban.
Commits diarios	La ausencia de al menos un commit significativo por cada día calendario previo a la entrega afecta el criterio de integración continua e historial Git.
Participación desigual	Ambos integrantes deben comprender el producto y demostrar contribuciones verificables.
Código generado con IA	El código que ninguno de los integrantes pueda explicar se considera contenido no dominado.
Pipeline fallido	Una integración continua en estado fallido impide considerar el incremento completamente terminado.
Incumplimiento del WIP	Debe registrarse y justificarse; ocultarlo o alterar evidencias constituye una falta de trazabilidad.

12. Lista de verificación antes de entregar

- ☐ El equipo está compuesto por exactamente dos estudiantes.
- ☐ El programa compila y puede ejecutarse.
- ☐ Las historias obligatorias y el cambio EXPEDITE están implementados.
- ☐ Las pruebas automatizadas pasan localmente y en la integración continua.
- ☐ Existe evidencia del ciclo RED → GREEN → REFACTOR.
- ☐ El tablero refleja el flujo real y respeta los límites WIP.
- ☐ Existe al menos un commit significativo por cada día calendario previo a la entrega.
- ☐ Ambos integrantes poseen commits significativos y pueden explicar el código.

- ☐ README.md, IA-LOG.md y RETROSPECTIVA.md están completos.
- ☐ La demostración técnica puede realizarse desde un entorno limpio.

Declaración de Responsabilidad

La entrega representa el trabajo y la comprensión de los dos integrantes. Toda asistencia de IA debe estar documentada, revisada y validada. La pareja es responsable de poder explicar las decisiones técnicas, las pruebas, el flujo de trabajo y cada componente incluido en el repositorio.